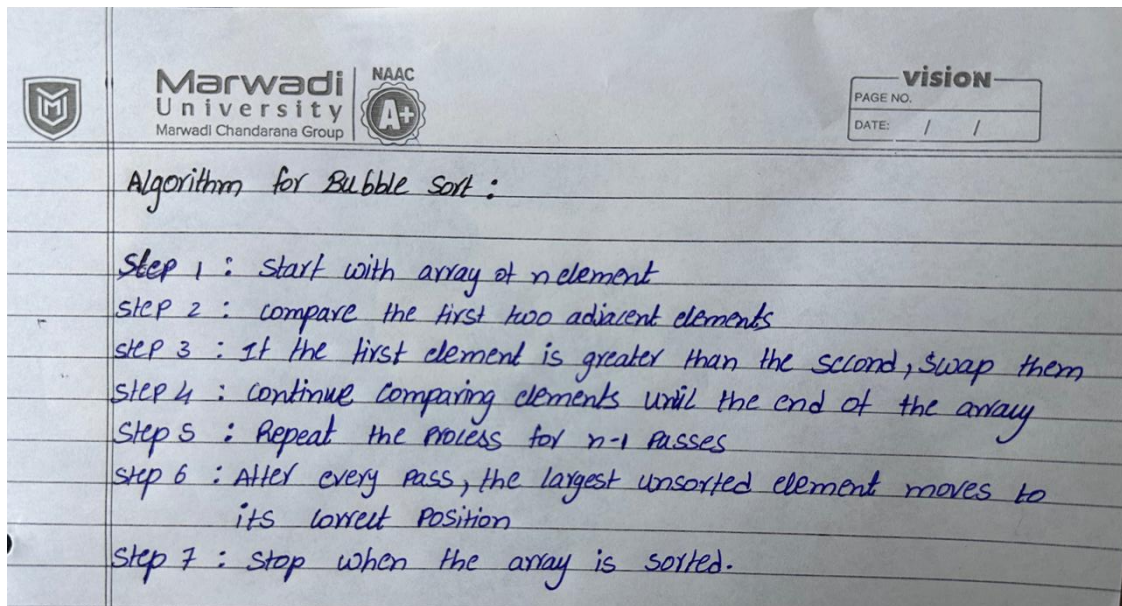


PRACTICAL - 1

AIM : Implementation and Time analysis of sorting algorithms, Bubble sort , Selection sort , Insertion sort , Merge sort and Quick sort.

ALGORITHM : // Bubble sort



PROGRAM : // Bubble sort

```
#include <iostream>
using namespace std;

// Function to perform Bubble Sort
void bubbleSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                // Swap the elements
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
}  
}  
}  
  
int main()  
{  
    int n;  
  
    // Enter the number of elements  
    cout << "Enter the number of elements: ";  
    cin >> n;  
  
    int arr[n];  
  
    // Enter array elements  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++)  
    {  
        cin >> arr[i];  
    }  
  
    bubbleSort(arr, n);  
  
    // Display sorted array  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

OUTPUT :

```
Enter the number of elements: 4  
Enter 4 elements: 12 0 8 11  
Sorted array: 0 8 11 12  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```



TIME COMPLEXITY : // Bubble sort

Time complexity:

```

for (i=0; i<n-1; i++) {
    for (j=0; j<n-1; j++) {
        if (A[j] > A[j+1]) {
            swap (A[j], A[j+1]);
        }
    }
}

```

$T(n) = n \times n = n^2 = O(n^2)$
 Best case = $O(n^2) \rightarrow$ No swap required
 Average case:
 $T(n) = (n-1) + (n-2) + \dots + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

$i=0 \rightarrow n-1$
 $i=1 \rightarrow n-2$
 $i=2 \rightarrow n-3$
 $\vdots$
 $i=n-2 \rightarrow 1$

Marwadi University
Marwadi Chandarana Group

NAAC
A+

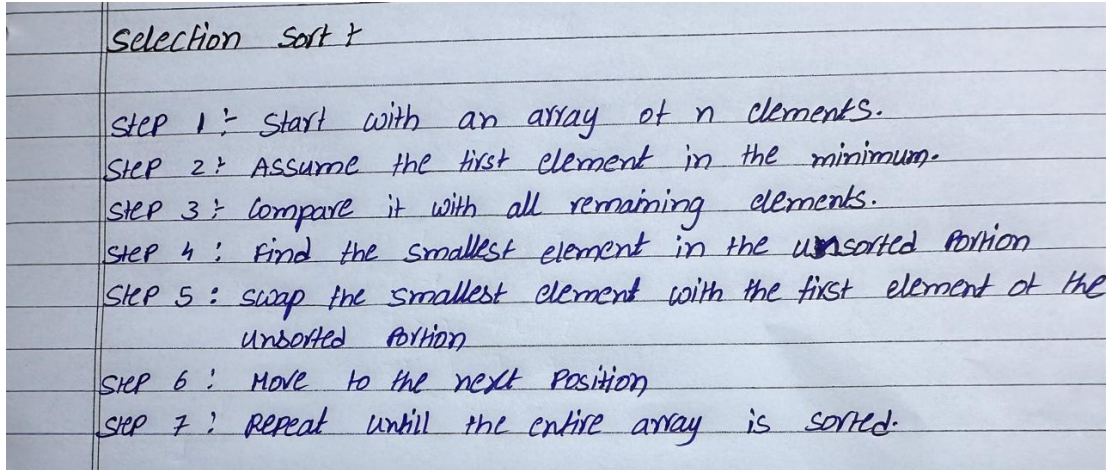
Vision
PAGE NO. _____
DATE: / /

Worst case:

$T(n) = (n-1) + (n-2) + \dots + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = \frac{n^2 - n}{2}$
 $T(n) = O(n^2) \Rightarrow$ Ignoring constants & lower-order terms

$i=0 \rightarrow n-1$
 $i=1 \rightarrow n-2$
 $\vdots$
 $i=n-2 \rightarrow 1$

ALGORITHM : // Selection sort



Selection Sort :-

- Step 1 :- Start with an array of n elements.
- Step 2 :- Assume the first element in the minimum.
- Step 3 :- Compare it with all remaining elements.
- Step 4 :- Find the smallest element in the unsorted portion
- Step 5 :- swap the smallest element with the first element of the unsorted portion
- Step 6 :- Move to the next position
- Step 7 :- Repeat until the entire array is sorted.

PROGRAM // Selection sort

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;

        // Find the smallest element in the remaining array
        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }

        // Swap the smallest element with the current element
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }
}

int main()
```



```
{  
    int n;  
  
    // Enter the number of elements  
    cout << "Enter the number of elements: ";  
    cin >> n;  
  
    int arr[n];  
  
    // Enter array elements  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++)  
    {  
        cin >> arr[i];  
    }  
  
    selectionSort(arr, n);  
  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

OUTPUT :

```
Enter the number of elements: 4  
Enter 4 elements: 12 21 9 0  
Sorted array: 0 9 12 21  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```




TIME COMPLEXITY : // Selection sort

Time complexity :

```

for (int i=0 ; i<n-1 ; i++) {
    int minIndex = i ;
    for (int j=i+1 ; j<n ; j++) {
        if (A[j] < A[minIndex]) {
            swap (A[i] , A[minIndex]);
        }
    }
}

```

Marwadi University
Marwadi Chandarana Group

NAAC
A+

vision
PAGE NO. _____
DATE: / /

Best case :

$$T(n) = (n-1) + (n-2) + (n-3) + \dots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

$i = 0 \rightarrow n-1$
 $i = 1 \rightarrow n-2$
 $i = 2 \rightarrow n-3$
 $\vdots$
 $i = n-2 \rightarrow 1 \text{ comparison}$

Average case :

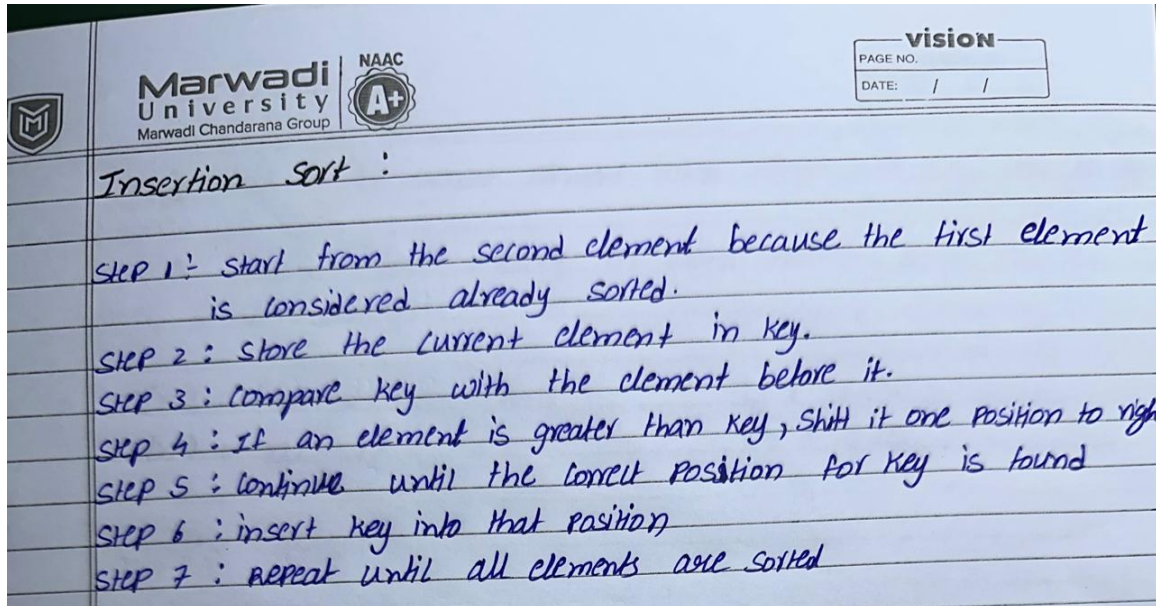
$$T(n) = 1+2+3+\dots+(n-1)$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = O(n^2)$$

worst case : $T(n) = O(n^2)$

ALGORITHM // Insertion sort



PROGRAM : // Insertion sort

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];

        // Compare with elements before it
        int j = i - 1;

        // Shift larger elements one position to the right
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        // Insert the key at its correct position
        arr[j + 1] = key;
    }
}
```

```
int main()
{
    int n;

    // Enter the number of elements
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    // Enter array elements
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    insertionSort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}
```

OUTPUT :

```
Enter the number of elements: 4
Enter 4 elements: 12 0 2 24
Sorted array: 0 2 12 24

...Program finished with exit code 0
Press ENTER to exit console.
```




TIME COMPLEXITY : Insertion sort

Time complexity :

```
for (int i = 1; i < n; i++) {
    int key = A[i];
    int j = i - 1;
    while (j >= 0 && A[j] > key) {
        A[j+1] = A[j]; j--;
    }
    A[j+1] = key;
}
```

Best case : for every element, we only need one comparison.

$$1 + 1 + 1 + \dots = n - 1$$

$$T(n) = n - 1$$

$$T(n) = O(n)$$

$$i = 1 \quad n \rightarrow 1$$

$$i = 2 \quad n \rightarrow 1$$

$$i = 3 \quad n \rightarrow 1$$

$$i = 4 \quad n \rightarrow 1$$

$\therefore$ There are no shifting operations.

Average case : The current element moves about half way through the sorted portion.
• some elements are already in correct position, while others need to be shifted.

$$T(n) = 1 + 2 + 3 + \dots + (n-1)$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

$$i = 1 \rightarrow 1/2$$

$$i = 2 \rightarrow 2/2$$

$$i = 3 \rightarrow 3/2 \dots$$

worst case : Reverse order. maximum shifting

$$T(n) = 1 + 2 + 3 + \dots + (n-1)$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

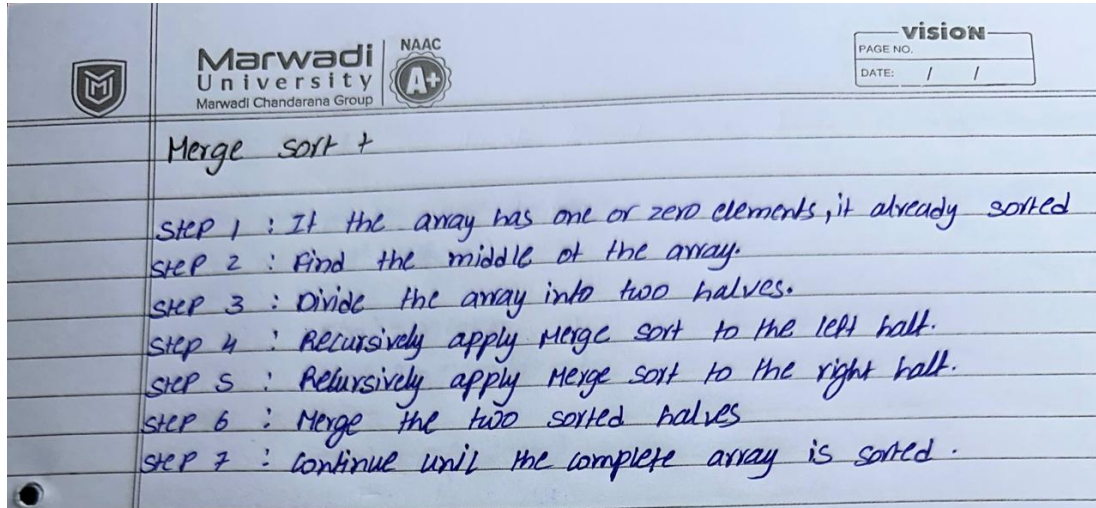
$$i = 1 \quad n \rightarrow 1$$

$$i = 2 \quad n \rightarrow 2$$

$$i = 3 \quad n \rightarrow 3$$

$$i = 4 \quad n \rightarrow 4$$

ALGORITHM : // Merge sort



PROGRAM : // Merge sort

```
#include <iostream>
using namespace std;

void merge(int arr[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;
    int k = 0;

    int temp[high - low + 1];

    // Compare elements from both parts
    while (i <= mid && j <= high)
    {
        if (arr[i] <= arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
```

```
}

// Copy remaining elements from the left part
while (i <= mid)
{
    temp[k] = arr[i];
    i++;
    k++;
}

// Copy remaining elements from the right part
while (j <= high)
{
    temp[k] = arr[j];
    j++;
    k++;
}

// Copy sorted elements back into the original array
for (i = low, k = 0; i <= high; i++, k++)
{
    arr[i] = temp[k];
}
}

// Function to perform Merge Sort
void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = low + (high - low) / 2;

        // Divide the array into two parts
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);

        // Merge the sorted parts
        merge(arr, low, mid, high);
    }
}

int main()
{
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;
```



```
int arr[n];

// Enter array elements
cout << "Enter " << n << " elements: ";
for (int i = 0; i < n; i++)
{
    cin >> arr[i];
}

mergeSort(arr, 0, n - 1);

cout << "Sorted array: ";
for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
}

return 0;
}
```

OUTPUT :

```
Enter the number of elements: 4
Enter 4 elements: 0 8 2 4
Sorted array: 0 2 4 8

...Program finished with exit code 0
Press ENTER to exit console.
```




TIME COMPLEXITY : // Merge sort

Time complexity +

```
void mergesort (int A[], int low, int high) {  
    if (low < high) {  
        int mid = (low + high) / 2;  
  
        mergesort (A, low, mid);  
        mergesort (A, mid + 1, high);  
        merge (A, low, mid, high);  
    }  
}
```

Best case +

$$T(n) = \log_2 n$$
$$\text{No. of levels} = O(\log n)$$
$$\text{each level takes} = O(n)$$
$$T(n) = O(n) \times O(\log n)$$
$$T(n) = O(n \log n)$$

Marwadi University
Marwadi Chandarana Group

NAAC
A+

vision
PAGE NO. _____
DATE: / /

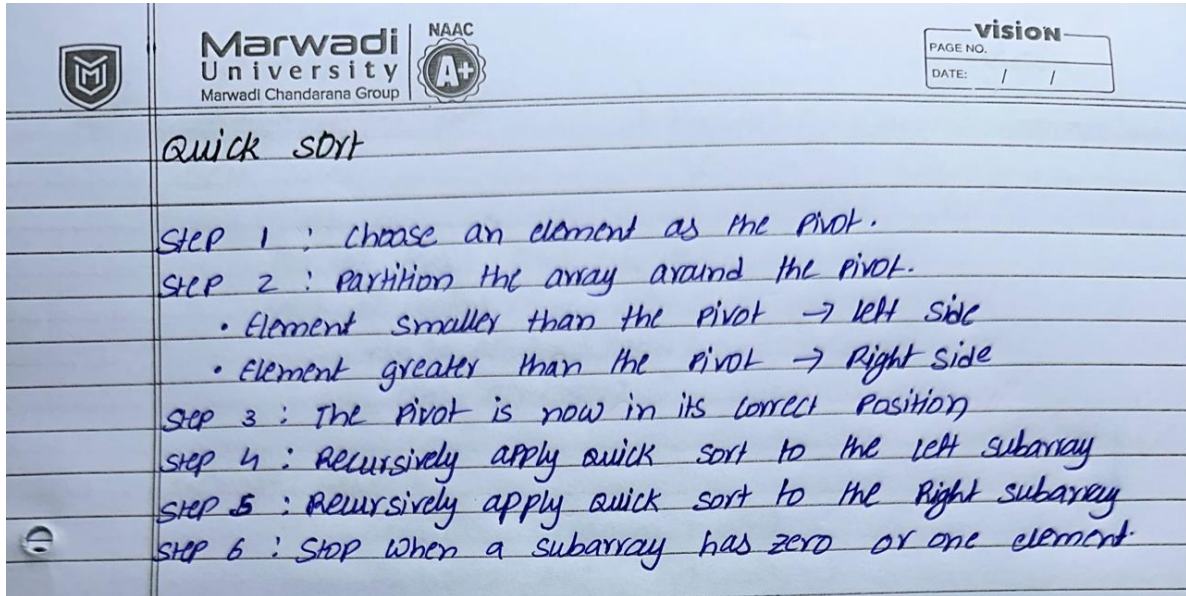
Average case +

$$\text{No. of levels } T(n) = \log n$$
$$\text{work at each } T(n) = n$$
$$T(n) = n \log n$$
$$T(n) = O(n \log n)$$

worst case +

$$T(n) = O(n \log n)$$

ALGORITHM : // Quick sort



PROGRAM : // Quick sort

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    // on the left side
    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
```

```
    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        // Find the pivot position
        int pivotIndex = partition(arr, low, high);

        // Sort the left part
        quickSort(arr, low, pivotIndex - 1);

        quickSort(arr, pivotIndex + 1, high);
    }
}

int main()
{
    int n;

    // Enter the number of elements
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    // Enter array elements
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    // Call Quick Sort function
    quickSort(arr, 0, n - 1);

    // Display sorted array
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}
```



OUTPUT :

```
Enter the number of elements: 4
Enter 4 elements: 21 12 36 0
Sorted array: 0 12 21 36

...Program finished with exit code 0
Press ENTER to exit console.
```

TIME COMPLEXITY : // Quick sort

Time complexity :-

```
void quicksort (int A[], int low, int high)
{
    if (low < high)
    {
        int p = partition (A, low, high);

        quicksort (A, low, p-1);
        quicksort (A, p+1, high);
    }
}
```

Marwadi University
Marwadi Chanderana Group

NAAC
A+

vision
PAGE NO.
DATE: / /

Best case :-

No. of levels $T(n) = \log_2 n$
work at each $T(n) = O(n)$
 $T(n) = O(n) \times O(\log n)$
 $T(n) = O(n \log n)$

Average case :-

partition operation = $O(n)$
recursion has $T(n) = O(\log n)$
 $T(n) = O(n \log n)$

Worst case :-

$T(n) = n + (n-1) + (n-2) + \dots + 1$
 $T(n) = \frac{n(n+1)}{2}$
 $T(n) = \frac{n^2 + n}{2}$
 $T(n) = O(n^2)$